SESSION 04 / PRINTABLE GUIDE

Session 04: Two dataset tiers

Use the **scenario** tier for the main participant lab. Keep the two small files for predicting individual transformations and explaining acceptance rules.

Tier	Inputs	Purpose
Micro fixtures	device_readings.csv, orders.jsonl	Nine events and four order documents; step through every row using lessons 02-07
Main lab	scenario/	Explore patterns, review sampled preparation, clean complete batches, and publish meaningful Warehouse Gold

Complete the mandatory pre-delivery setup first, then use the realistic notebook walkthrough. The presenter must check completion with the customer 3-5 business days before delivery; uploading data and fixing permissions are not delivery-day exercises. The micro lessons and main lab use **different table and batch names**. Never mix their input folders, expected counts, or Warehouse publication scripts.

What the main datasets contain

Telemetry: 50 devices, five sites, three device types, October 1-14, 2026, on a five-minute schedule. Fourteen CSV files contain **201,579 source rows**: 194,545 accepted events and 7,034 quarantined rows. The daily output has 700 device/day rows. Temperature has a daily shape, device-type differences, and a deliberately injected increase associated with the S03 firmware change at October 7, 12:00 UTC. That is a synthetic association, not evidence of causality.

Of 201,600 scheduled slots, 1,026 have no source event. There are also 1,005 extra duplicate occurrences. These quantities differ from rejection rows and accepted coverage. Missing intervals include a one-hour D017 gap on October 7. Quality problems cover missing keys, invalid timestamps, nonnumeric/blank temperatures, out-of-range readings, and repeated event keys. Signal strength is optional and is missing in 1,002 rows; null does not mean healthy or zero signal.

Orders: 5,000 nested documents across fourteen JSON Lines files, 100 synthetic customers, four regions, twelve SKUs, four product categories, and four order statuses. Documents have variable line counts, optional shipping structures, and optional product descriptions. Array expansion yields **15,054 evidence rows**, including 50 empty-order markers. The contract accepts 14,391 lines, quarantines 663 evidence rows, and totals **5,643,227.95 in accepted-line amount**. This includes multiple statuses, including cancelled orders: it is **not recognized revenue or a complete-order ledger**.

The rejection rates are approximately 3.49% of raw device rows and 4.40% of expanded order evidence. Source omissions are not quarantine records. These are manageable training datasets, not a demonstration that Spark is faster than SQL or required by their size.

Folder contract and reproducibility

```text
scenario/
  devices/             14 source CSV files, identical 11-column header/order
  orders/              14 source JSON Lines files, one document per line
  expected/            Complete approved transfer results; never ingest as sources
    device_daily.csv   700 expected device/day rows
    order_lines.csv    14,391 expected accepted order lines
  manifest.json        Seeds, exact counts, quality totals, and SHA-256 file hashes
```

All records are synthetic; no customer systems or external downloads are used. The approximately 23 MB of materialized files are included so participants need not run a generator. Before delivery, upload only `devices/` and `orders/` to the approved ADLS Gen2 source, then copy them unchanged into the matching `Files/session_04/scenario` folders. Upload `manifest.json` and `expected/` separately as teaching assets, following the prerequisite guide. Direct upload of the entire `scenario` folder is an explicitly approved fallback, not the default source-ingestion path. Read only the explicit `devices/*.csv` or `orders/*.jsonl` source globs, never the whole scenario tree.

Maintainers can reproduce the shipped bytes using Python's standard library:

```bash
python lessons/session_04/artifacts/generate_datasets.py
python lessons/session_04/artifacts/validate_examples.py
```

The generator overwrites only its named dataset files, not the micro fixtures. Do not regenerate an input folder while a notebook/copy is using it. The offline validator independently recalculates acceptance and all expected transfer rows, checks file hashes, and tests deterministic regeneration in a temporary folder. These checks do not execute Fabric/Spark, Data Wrangler, SQL, or copy activities.

The manifest/expected files are a **teaching answer key**, not a production ingestion manifest or a mechanism for establishing that an upstream export is complete. Both notebook tracks use current-batch replacement, not incremental processing or an all-batch history.

Source: artifacts/data/README.md. Companion-material links open the repository originals.